AST AUTOMATION PLAN

TECH STACK CONFLICT ANALYSIS

Identified conflicts between the v2.0 Implementation Plan and actual client tech stacks | March 2026

4	5	2	1
CRITICAL	HIGH	MEDIUM	LOW

Overview

Cross-referencing the v2.0 RPA Implementation Plan against the Master Resource List revealed 13 conflicts across 5 of the 6 clients. These range from critical system mismatches (wrong call platform assumed) to missing systems that need dedicated automation profiles.

No conflicts were found for Legacy's core systems — CTM, Salesforce, and ZOHO are all correctly referenced in the plan. Legacy has one medium-severity note about ZOHO SalesIQ needing a specific DOM profile.

Conflict Summary Table

Client	Conflict	Severity	Resolution Direction
TBT	No CTM — SF Lightning Only	**CRITICAL**	Retarget all call automation to SF Lightning DOM
TBT	Shared SF Instance with Flyland	**HIGH**	Add owner/record-type filters to all SF reads/writes
TAKAHAMI	RingCentral, not CTM	**CRITICAL**	Rebuild call triggers around RingCentral web app DOM
TAKAHAMI	CollaborateMD not in plan	**CRITICAL**	Build CMD-specific RPA profile for claims/status/appeals
TAKAHAMI	KIPU EHR missing from plan	**HIGH**	Add KIPU as fallback data source for clinical fields
EMB	Adobe Acrobat, not pdfplumber	**CRITICAL**	Use Acrobat JS macros or PyAutoGUI for form-filling
EMB	Elevate billing portal missing	**HIGH**	Add Elevate as primary data source / write-back target
EMB	Availity + VerifyTx are specific portals	**HIGH**	Build dedicated RPA profiles for each portal
EMB	TurboScribe already handles STT	**MEDIUM**	Remove STT build; use TurboScribe exports as input
BANYAN	Tracker is Google Form, not web app	**HIGH**	Use Google Form URL pre-fill or direct Sheets write
BANYAN	Ticketing is Power Apps	**HIGH**	Use ARIA selectors + slow-type for Power Apps inputs
LEGACY	ZOHO SalesIQ needs specific profile	**MEDIUM**	Build SalesIQ DOM profile for chat extension
FLYLAND	ZOHO Mail differs from SalesIQ	**LOW**	Separate ZOHO Mail DOM profile from SalesIQ profile

Detailed Conflict Findings

TBT — TBT Has No CTM — Uses Salesforce Lightning as Primary System

CRITICAL

What the plan assumed	Plan assumes CTM (calltrackingmetrics.com) for call event detection, DOM monitoring, and outbound dial automation in TBT.
What actually exists	TBT's listed 'CTM' URL resolves to https://wethebest.lightning.force.com — a Salesforce Lightning instance, not Call Tracking Metrics. TBT does not appear to use CTM at all.
Impact on plan	All TBT automations that hook into CTM DOM (lead routing, call detection, wrap-up triggers) will have no target system. The entire call-tracking layer needs to be rebuilt around Salesforce Lightning.
Recommended fix	Re-target all TBT call-event automations to Salesforce Lightning DOM (wethebest.lightning.force.com). Use Salesforce Lightning's activity timeline and lead views as the data source instead of CTM. Chrome extension selectors need to be written for SF Lightning UI, not CTM.

Specific action items:

- Auto-Lead Pruning: CSV export trigger should come from SF Lightning's lead list view, not CTM
- Auto-Lead/Number Pull & Routing: read incoming activity from SF Lightning dashboard, not CTM call notification
- Note Templates: disposition triggers should fire on SF Lightning activity record save, not CTM wrap-up

TBT — TBT and Flyland Share the Same Salesforce Instance

HIGH

What the plan assumed	Plan treats Flyland and TBT as having separate, independent Salesforce CRM environments.
What actually exists	Both Flyland and TBT point to the same SF org: flyland.my.salesforce.com. TBT's 'CRM' entry is the same URL as Flyland's.
Impact on plan	Any automation that creates, updates, or reads SF records for TBT will be operating inside Flyland's SF instance. Data scoping must be enforced carefully to avoid cross-client record contamination. Shared SF also means one Chrome extension config can serve both — reducing deployment work but increasing risk.
Recommended fix	Add record-type or owner-based filters to all SF DOM reads for TBT. Confirm with client whether TBT uses a separate SF account owner or record type to distinguish their leads. Update field-mapping configs to include a client/team identifier filter.

Specific action items:

- SF DOM scraper must filter by TBT-specific owner, account type, or custom field before reading/writing
- New lead auto-creation must assign to correct TBT owner/queue to avoid mixing with Flyland records
- Single Chrome extension deployment can cover both clients if selector configs are kept separate per subdomain path

TAKAHAMI — Takahami Uses RingCentral, Not CTM — Completely Different Call System

CRITICAL

What the plan assumed	Plan references CTM DOM monitoring for call events and assumes a generic 'online fax portal' for appeal submissions.
What actually exists	Takahami uses RingCentral (ringcentral.com) as their calling app — no CTM at all. RingCentral also has built-in fax capability via its web interface.
Impact on plan	All call-event triggers in the plan target CTM DOM elements that don't exist in Takahami's stack. The fax workaround (navigating to a separate fax portal) is unnecessary — RingCentral's own web app can send faxes.
Recommended fix	Rewrite Takahami's call-event detection to monitor RingCentral web app DOM (app.ringcentral.com). For fax submissions, automate directly within RingCentral's fax interface using Playwright — the agent is already logged in. This is actually simpler than using a third-party fax portal.

Specific action items:

- Auto-Fax for Appeals: use Playwright to navigate RingCentral web → Fax → attach document → fill recipient from local carrier directory JSON → send
- Claim Status Updater: target CollaborateMD DOM (see conflict below), not a generic billing portal
- Negotiation Reply Suggester: read carrier responses from email client open in browser (Outlook/Gmail), not a generic message thread

TAKAHAMI — CollaborateMD is the Billing/Claims System — Not in Plan

CRITICAL

What the plan assumed	Plan refers to a generic 'billing portal' for claim lookups and status changes without naming the specific system.
What actually exists	Takahami uses CollaborateMD (CMD) at app.collaboratemd.com for all billing, claims follow-up, and status management. This is a specialized medical billing platform with its own specific DOM structure.
Impact on plan	The Claim Status Updater, Appeal Router, and Negotiation Form-Filler all need to operate inside CollaborateMD's UI. Generic selectors will not work. CollaborateMD has specific claim list views, status dropdowns, and appeal workflows that need dedicated Playwright/extension profiles.
Recommended fix	Build a CollaborateMD-specific RPA profile: map the DOM selectors for claim list view, claim detail fields (billed amount, allowed amount, status dropdown), and appeal submission flow. Store as a named portal profile in the extension config.

Specific action items:

- Claim lookup: target CollaborateMD claims follow-up page DOM for billed/allowed amount fields
- Status change: find and interact with CollaborateMD's status dropdown (PAID → UNDERPAID) via DOM
- Appeal Router: read claim type from CollaborateMD claim detail page, not a generic portal
- Cheat Sheet (Excel on SharePoint): use this existing resource as the carrier fax directory instead of building a new JSON file

TAKAHAMI — KIPU EHR System Not Accounted For in Plan

HIGH

What the plan assumed	Plan does not reference any EHR (Electronic Health Record) system for Takahami.
What actually exists	Takahami uses KIPU EHR (lhc10828.kipuworks.com) alongside CollaborateMD. Patient clinical data lives in KIPU, while billing/claims live in CollaborateMD. Negotiation and appeal work may require cross-referencing both.

Impact on plan	Negotiation form-filling and claim lookup automations may need to pull patient data from KIPU if the CollaborateMD record doesn't contain all required fields. Missing this system could result in incomplete form fills.
Recommended fix	Add KIPU as a secondary data source for Takahami automations. When negotiation form-filler runs, check if required fields (e.g., diagnosis codes, treatment dates) are available in CollaborateMD first; if not, trigger a KIPU DOM lookup as a fallback. Build a KIPU portal profile in the extension config.

Specific action items:

- Negotiation Form-Filler: add KIPU fallback lookup for clinical fields not present in CollaborateMD
- KIPU DOM profile: map patient lookup by name/ID, extract relevant clinical fields
- Carrier fax directory: already exists in the SharePoint Cheat Sheet — scrape that instead of building from scratch

EMB — EMB Uses Adobe Acrobat — Not pdfplumber/pdfrw

CRITICAL

What the plan assumed	Plan proposes using Python libraries (pdfplumber, pdfrw) to programmatically read and write PDF form fields for the PDF Auto-Filler.
What actually exists	Element Medical Billing uses Adobe Acrobat (both desktop and web at adobe.com). Specialists open, fill, and save PDFs inside Acrobat — not in a browser or via Python scripts.
Impact on plan	The PDF auto-fill approach in the plan won't integrate with Acrobat's workflow. Agents don't open PDFs in a browser — they open them in Acrobat desktop app. Python file-write approach would create a separate file the agent hasn't opened, breaking their workflow.
Recommended fix	Two viable workarounds: (1) Use PyAutoGUI to detect when Acrobat opens a PDF, then simulate Tab/field navigation and typing within Acrobat's form fill interface. (2) Use Acrobat's built-in JavaScript API (Acrobat supports JS macros) to auto-fill fields when a form opens — no external tool needed, runs inside Acrobat itself. Option 2 is more stable.

Specific action items:

- Primary approach: Acrobat JavaScript action that fires on form open, reads patient data from clipboard or a local data file, and auto-populates standard fields
- Fallback: PyAutoGUI desktop automation that detects Acrobat window, maps field tab-order, and types in values
- pdfplumber/pdfrw can still be used for the Template Generator (generating blank template files), but not for live form-filling inside Acrobat

EMB — Elevate Billing Portal Not in Plan

HIGH

What the plan assumed	Plan does not mention Elevate as a system for EMB. No automation targets it.
What actually exists	EMB uses Elevate (elevate.serverdata.net) as their billing system. This is likely where VOB results and patient billing records are stored and updated post-verification.
Impact on plan	Post-call system updates and data sources for PDF auto-fill may need to pull from or write to Elevate, not just SF. If Elevate is the primary data store (not SF), all DOM-scraping for patient data needs to target Elevate's interface.

Recommended fix	Add Elevate as a data source portal in the RPA stack. Build a Playwright/extension profile for Elevate's patient lookup and record update flows. Determine with client whether Elevate or SF is the primary patient data source for form-filling — this changes where the extension reads from.

Specific action items:

- PDF Auto-Filler: if patient data lives in Elevate (not SF), DOM scraper must target Elevate's patient detail page instead
- Portal Integration: add Elevate to the list of systems the wrap-up sync writes back to
- Elevate profile: map DOM selectors for patient ID lookup, insurance fields, and record update flow

EMB — Availity and VerifyTx Are the Specific Insurance/Verification Portals

HIGH

What the plan assumed	Plan references 'insurance portals' and 'verification portals' generically, proposing Selenium scraping with URL-pattern detection.
What actually exists	EMB specifically uses Availity (availity.com) for insurance lookups and VerifyTx (app.verifytx.com) for benefits verification. These are the two concrete portals that need RPA profiles.
Impact on plan	Generic portal profiles won't work — each portal has unique DOM structure, login flows, and result page layouts. Without Availity and VerifyTx-specific selector maps, the scraper won't know where to find eligibility data.
Recommended fix	Build dedicated Playwright/extension profiles for Availity and VerifyTx. Map the eligibility lookup flow, result fields (deductible, copay, coverage status), and cache structure for each. These become the two concrete portal targets replacing the generic 'insurance portal' reference in the plan.

Specific action items:

- Availity profile: map eligibility search page, member ID field, and coverage result DOM selectors
- VerifyTx profile: map verification request flow and benefit result extraction
- Cache layer: store results per policy ID with 24hr expiry in localStorage
- Reminder engine: trigger 'Check deductible' and 'Check out-of-network' pop-ups based on Availity/VerifyTx result fields

EMB — TurboScribe Already Handles Transcription — Don't Rebuild STT

MEDIUM

What the plan assumed	Plan proposed building or integrating a speech-to-text solution (Google STT or local model) for note generation and call summarization.
What actually exists	EMB already uses TurboScribe (turboscribe.ai) for transcription. Agents already have a working transcription pipeline.
Impact on plan	Building a separate STT layer is wasted effort. More importantly, TurboScribe outputs are likely already saved as text files or accessible via the TurboScribe dashboard — these outputs can be used directly as input to the note summarizer and auto-fill automations.
Recommended fix	Remove STT from EMB's scope. Instead, build the note summarizer to ingest TurboScribe's output: (1) File watcher monitors TurboScribe's download/export folder for new transcript files, (2) Local NLP model summarizes the transcript text, (3) Summary injected into Elevate/SF note field. No STT development needed.

Specific action items:

- File watcher: monitor TurboScribe export folder for new .txt/.docx transcript files
- Auto-Note: feed TurboScribe transcript into local summarizer (DistilBERT) instead of building STT
- Remove Google STT and LibreTranslate from EMB's deployment requirements

BANYAN — Banyan's Tracker is a Google Form — Cannot Be Auto-Filled Like a Web App

HIGH

What the plan assumed	Plan assumes the call tracker is a tab-based web interface (like a spreadsheet or internal dashboard) that can be updated via DOM injection or simulated input after each call.
What actually exists	Banyan's tracker is a Google Form (specific URL: docs.google.com/forms/…). Google Forms are submit-only inputs — they cannot be pre-populated via URL parameters in the standard embed and each submission creates a new response row.
Impact on plan	The 'multi-tab wrap-up sync' approach (navigating to tracker tab and filling fields) still works, but Google Forms have specific DOM structures. The form fields are rendered differently from a regular web app, and the submit action creates a new row in the linked Google Sheet — which is actually what we want.
Recommended fix	The Google Form can be auto-filled using URL query parameters (Google Forms supports ?entry.FIELD_ID=VALUE pre-fill via URL). Build the automation to construct a pre-filled Google Form URL from wrap-up data, open it in a tab, and auto-submit — cleaner than DOM injection. Alternatively, write directly to the linked Google Sheet via Playwright if agents have edit access.

Specific action items:

- Wrap-up auto-sync: construct pre-filled Google Form URL using field entry IDs + wrap-up data, open and auto-submit
- Map Banyan Tracker form field entry IDs (inspectable from form source) to CTM wrap-up fields
- Alternative: if agents have Google Sheets edit access, Playwright can write directly to the linked Sheet, bypassing the form entirely
- Either approach eliminates manual form submission without needing Google Sheets API

BANYAN — Banyan's Ticketing is Power Apps — Complex DOM, Not a Standard Web Form

HIGH

What the plan assumed	Plan refers to 'admission ticket form' generically, assuming standard HTML form fields that can be auto-filled via DOM manipulation or simulated input.
What actually exists	Banyan uses a Power Apps canvas app (apps.powerapps.com) for ticketing. Power Apps renders controls as custom elements — not standard HTML inputs — making DOM injection unreliable. Fields are rendered in an iframe-based canvas with custom event handling.
Impact on plan	Standard DOM injection and simulated typing are less reliable in Power Apps. Power Apps controls intercept keyboard events differently from normal HTML inputs, which can cause automation to miss fields or trigger validation errors.
Recommended fix	Use PyAutoGUI (coordinate-based clicking and typing) as the primary approach for Power Apps, since DOM manipulation is unreliable. Alternatively, use Playwright with a slow-type approach (character-by-character input with delays) which Power

	Apps controls tend to accept. Map field positions or use accessibility tree (ARIA labels) as selectors instead of CSS selectors.

Specific action items:

- Power Apps form-fill: use Playwright with accessibility tree selectors (aria-label attributes) rather than CSS DOM selectors
- Slow-type mode: type characters with 50ms delay between keystrokes to avoid Power Apps input race conditions
- Fallback: PyAutoGUI coordinate-based clicking for fields where ARIA selectors are unavailable
- Pre-fill data source: read caller info from the SF pop-up overlay already shown during the call

LEGACY — Legacy's Chat System is ZOHO SalesIQ — Needs Specific DOM Profile

MEDIUM

What the plan assumed	Plan refers to 'chat window DOM' and 'active chat window content' generically for the chat reply suggester.
What actually exists	Legacy uses ZOHO SalesIQ (zoho.com/salesiq) for chat — a specific platform with its own DOM structure, message thread layout, and input field implementation. Flyland also uses ZOHO but via ZOHO Mail/Chat (different product).
Impact on plan	The Chrome extension's chat reader needs ZOHO SalesIQ-specific CSS selectors to find the message thread and input field. Generic selectors won't work across different chat platforms.
Recommended fix	Build a ZOHO SalesIQ-specific DOM profile for the chat reply suggester extension. Map the CSS selectors for: incoming message container, conversation thread, and reply input field. This is a straightforward selector mapping — ZOHO SalesIQ's web app has inspectable, stable DOM classes.

Specific action items:

- Extension config: add 'zoho.com/salesiq' as a recognized URL pattern with SalesIQ-specific selectors
- Flyland uses ZOHO Mail/Chat (different product) — needs a separate selector profile for zoho.com/mail
- Reply suggestion logic is identical — only the DOM selectors differ between SalesIQ and ZOHO Mail

FLYLAND — Flyland Uses ZOHO Mail/Chat — Different From ZOHO SalesIQ Used by Legacy

LOW

What the plan assumed	Plan groups chat automation generically without distinguishing between ZOHO products.
What actually exists	Flyland uses ZOHO Mail (zoho.com/mail) for chat, while Legacy uses ZOHO SalesIQ — these are different ZOHO products with different DOM structures.
Impact on plan	Minor — both are ZOHO products accessible via browser, but the chat thread DOM and input field selectors will differ between ZOHO Mail and ZOHO SalesIQ.
Recommended fix	Create separate DOM selector profiles for ZOHO Mail (Flyland) and ZOHO SalesIQ (Legacy) in the Chrome extension config. The suggestion logic is shared — only the selectors are client-specific.

Specific action items:

- Extension: two ZOHO profiles — one for zoho.com/mail (Flyland), one for zoho.com/salesiq (Legacy)

- Shared suggestion engine; client-specific DOM mapping configs

Revised System Map by Client

Updated reference showing actual tools per client and their automation role after resolving conflicts.

Flyland Recovery

System	Tool	Automation Role	Status
Call Tracking	CTM (calltrackingmetrics.com)	Call event trigger, DOM monitoring for incoming calls	✓ Confirmed
CRM	Salesforce (flyland.my.salesforce.com)	Account pop-up, note injection, lead creation — shared with TBT, apply data filters	⚠️ Shared w/ TBT
Chat	ZOHO Mail/Chat (zoho.com/mail)	Chat reply suggester — needs ZOHO Mail-specific DOM profile	⚠️ Needs profile

TBT

System	Tool	Automation Role	Status
Call / CRM	Salesforce Lightning (wethebest.lightning.force.com)	Call activity detection, lead routing, wrap-up triggers — NO CTM	✗ Plan wrong
CRM (shared)	Salesforce (flyland.my.salesforce.com)	Shared SF org with Flyland — must filter by TBT owner/record type	⚠️ Filter needed

Takahami

System	Tool	Automation Role	Status
Calling / Fax	RingCentral (ringcentral.com)	Call event triggers + fax submission via RingCentral web app — replaces CTM and third-party fax	✗ Plan wrong
Billing / Claims	CollaborateMD (app.collaboratemd.com)	Claim lookup, status update, appeal workflow — needs dedicated RPA profile	✗ Not in plan
EHR	KIPU (lhc10828.kipuworks.com)	Clinical data fallback for negotiation form-filling	✗ Not in plan
Reference	Cheat Sheet (Excel on SharePoint)	Use as carrier fax directory — replaces building a new JSON config	✓ Use existing
Document Mgmt	SharePoint (Takahami Billing)	Appeal document storage and retrieval for auto-fax	✓ Confirmed

Element Medical Billing

System	Tool	Automation Role	Status
PDF	Adobe Acrobat	Form-filling via Acrobat JS macros or PyAutoGUI — replaces pdfplumber/pdfrw for live fills	✗ Plan wrong
Billing	Elevate (elevate.serverdata.net)	Primary patient data source for form-filling and post-call record updates	✗ Not in plan
Insurance	Availity (availity.com)	Insurance eligibility lookup — primary portal RPA target	⚠️ Generic in plan
Verification	VerifyTx (app.verifytx.com)	Benefits verification portal — secondary portal RPA target	✗ Not in plan
Transcription	TurboScribe (turboscribe.ai)	Existing STT — use export files as input to note summarizer, no new STT needed	✓ Use existing
Document Mgmt	SharePoint + Office 365	Appeal document storage, template management	✓ Confirmed

Banyan

System	Tool	Automation Role	Status
Call Tracking	CTM (calltrackingmetrics.com)	Call event trigger, DOM monitoring	✓ Confirmed
CRM	Salesforce (banyan.lightning.force.com)	Account pop-up, record updates — separate SF instance from Flyland/TBT	✓ Confirmed
Tracker	Banyan Tracker (Google Form)	Pre-fill via URL params or direct Google Sheets write — not standard tab automation	⚠️ Approach change
Ticketing	Power Apps (apps.powerapps.com)	Use ARIA selectors + slow-type mode — not standard DOM injection	⚠️ Approach change
Reporting	Google Sheets	Reporting data source, Playwright write target	✓ Confirmed
Communication	MS Teams	Internal transfer alerts, escalation notifications	✓ Confirmed

Legacy

System	Tool	Automation Role	Status
Call Tracking	CTM (calltrackingmetrics.com)	Call event trigger, wrap-up monitoring	✓ Confirmed
CRM	Salesforce	History auto-pull, record updates	✓ Confirmed
Chat	ZOHO SalesIQ	Chat reply suggester — needs SalesIQ-specific	⚠️ Needs

System	Tool	Automation Role	Status
	(zoho.com/salesiq)	DOM profile	profile
Reporting	Google Sheets	Wrap-up sync write target	✓ Confirmed
Communication	MS Teams	Internal team notifications	✓ Confirmed